

Low-Latency AI Safety: Engineering Sub-30ms Guardrail Pipelines for Retrieval-Augmented Generation

In production Retrieval-Augmented Generation (RAG) architectures, real-time interactive performance demands strict latency limits¹. Applications such as real-time voice agents, enterprise search interfaces, and automated conversational assistants operate under rigid end-to-end Service Level Objectives (SLOs)¹. To preserve conversational flow and prevent jarring user pauses, total Time to First Token (TTFT) must remain minimal¹. When integrating safety guardrails to intercept harmful inputs, jailbreaks, and indirect prompt injections, the security infrastructure must perform its evaluation within a strict sub-30 millisecond (sub-30ms) execution window³.

Conventional AI safety paradigms often rely on autoregressive "LLM-as-a-Judge" models or multi-turn reasoning classifiers⁶. However, these approaches introduce latency overheads ranging from 100 milliseconds to several seconds, rendering them fundamentally unviable for inline real-time screening⁶. Achieving deterministic sub-30ms guardrail execution requires a complete departure from generative reasoning models in favor of lightweight discriminative classifiers, structural heuristics, compiled static runtimes, and optimized vector cache lookup strategies³.

Deconstructing the 30ms Execution Budget

To reliably disregard harmful prompts within 30ms under production conditions, system architects must disaggregate total request processing into discrete micro-operations². Latency is an additive pipeline sum consisting of transport, queue scheduling, tokenization, neural inference, and policy enforcement². Mathematically, total guardrail latency is expressed as:

$$\text{Total Guardrail Latency} = t_{\text{transport}} + t_{\text{queue}} + t_{\text{tokenize}} + t_{\text{inference}} + t_{\text{vector}} + t_{\text{orchestration}}$$

A production-grade 30ms budget allocates strict microsecond-level and millisecond-level constraints across each architectural tier². Ingress transport terminates TLS handshakes and parses JSON payloads at the edge, requiring minimal processing time². Tier 1 heuristic screening relies on compiled pattern matching to evaluate structural risk immediately³. Tier 2 neural inference handles sequence classification over bidirectional transformers, representing the primary compute phase³. Tier 3 vector caching computes truncated embeddings to query historical threat vector databases¹³. Finally, orchestration logic evaluates policy rules and dispatches allow or block decisions³.

Pipeline Stage	Operational	Target Latency	Target Latency
----------------	-------------	----------------	----------------

	Mechanics	Budget (P50)	Budget (P95)
Ingress Transport	Edge API gateway termination, TLS handshake, payload parsing	1.0 ms – 2.0 ms	3.0 ms
Tier 1: Heuristic Screening	Compiled Rust regex pattern matching, character n-gram lookup	0.2 ms – 0.8 ms	1.0 ms
Tier 2: Neural Discriminator	INT8 ONNX bidirectional transformer inference (e.g., DeBERTa-v3)	4.5 ms – 7.0 ms	12.0 ms
Tier 3: Vector & Cache Check	MRL-truncated embedding lookup (BGE-M3) against threat vector store	3.0 ms – 8.0 ms	12.0 ms
Orchestration & Decision	Threshold evaluation, policy mapping, decision dispatch	0.1 ms – 0.3 ms	0.5 ms
Total Pipeline Overhead	Inline pre-execution safety gate	8.8 ms – 18.1 ms	28.5 ms

Allocating budgets in this manner ensures that even under P95 tail-latency spikes caused by long input prompts or transient CPU contention, safety screening completes within the hard 30ms ceiling³.

Discriminative Architecture vs. Generative Reasoning Classifiers

A primary driver of high latency in safety moderation is the reliance on Chain-of-Thought (CoT)

reasoning models⁶. Prevailing designs generate step-by-step explanatory tokens prior to issuing a final binary verdict (SAFE or UNSAFE)⁶. While step-by-step reasoning improves complex multi-step logic, empirical research demonstrates that CoT yields zero net accuracy gain for short, non-symbolic safety classification tasks⁶.

The Fallacy of Chain-of-Thought Moderation

Empirical evaluations comparing reasoning-based decoders against label-only discriminative encoders reveal that reasoning tokens generated by safety decoders are predominantly post-hoc justifications⁶. Resampling experiments confirm that altering or removing the generated CoT explanation almost never shifts the final safety decision¹¹. The model determines the safety state during the initial forward passes across the prompt, rendering subsequent token generation redundant compute overhead⁶.

Controlled benchmarks comparing a 395M parameter label-only bidirectional encoder (LeanGuard) against larger autoregressive decoder guards demonstrate that single-pass classification over a 512-token context achieves equivalent moderation performance while eliminating autoregressive decoding⁶. The reduction in computational footprint is defined by:

$$\text{Compute Scale Factor} = \frac{\text{Decoder Auto-Regressive Tokens}}{\text{Encoder Single Forward Pass}} \approx 100\times$$

By removing autoregressive token generation, bidirectional encoders process the full input context concurrently via parallel matrix multiplications⁶. This structural shift reduces compute requirements by approximately 100x and drops inference latencies from hundreds of milliseconds down to sub-10ms regimes³.

Ultra-Compact Discriminators: Parameter Profiles and Performance

For sub-30ms budgets, modern production architectures leverage compact bidirectional transformer encoders fine-tuned explicitly on prompt injection, jailbreak, and content harm corpora³. Fine-tuned 70M parameter models such as DeBERTa-v3-xsmall utilize disentangled attention mechanisms to serve as hyper-efficient binary attack classifiers³. When compiled into quantized Open Neural Network Exchange (ONNX) format, these models execute warm CPU inferences in ~5ms at P50 and ~7ms at P95³.

Meta's Prompt-Guard 2 (86M parameters) provides specialized policy screening and jailbreak detection, delivering robust structural injection detection within single-digit millisecond windows when deployed via INT8 ONNX runtimes³. For edge deployments, models like ModernGuard-1 support context windows up to 2,048 tokens without exceeding sub-30ms pipeline budgets⁵. Similarly, LeanGuard (395M parameters) functions as a single-pass bidirectional encoder engineered to replace generative decoders entirely, reaching an average F1 score of 82.90% across standard public benchmarks over a 512-token context⁶.

Temperature Calibration for Probability Reliability

Raw classification logits produced by deep neural networks frequently exhibit uncalibrated

overconfidence, leading to erratic false-positive spikes³. To enforce consistent risk scoring without altering the underlying classification boundary at **0.5**, raw model logits must undergo scalar temperature scaling before entering the softmax activation layer³.

Given raw model output logits $z = [z_0, z_1]$ representing safe and attack states respectively, and a learned positive temperature scalar $T > 0$ derived by minimizing Negative Log-Likelihood (NLL) over a validation set, the calibrated probability is formulated as³:

$$P(\text{attack} \mid x) = \frac{\exp\left(\frac{z_1}{T}\right)}{\exp\left(\frac{z_0}{T}\right) + \exp\left(\frac{z_1}{T}\right)}$$

Temperature calibration smoothly compresses overconfident extreme probabilities, such as mapping raw 99.8% output noise down to a calibrated 82.0% probability³. This mathematical adjustment stabilizes downstream policy routing and thresholding without adding computational overhead³.

Cascaded Multi-Tier Guardrail Architecture

Single-model guardrail designs struggle to simultaneously optimize throughput, latency, and comprehensive policy coverage¹². To achieve sub-30ms operational guarantees, high-throughput systems employ a multi-tier cascaded architecture³. Fast, low-cost structural filters evaluate incoming prompts first, short-circuiting obvious attacks in under 1ms, while deeper neural models evaluate ambiguous or complex semantics³.

In a production execution cascade, an incoming request flows sequentially through three evaluation tiers before reaching the target LLM. The request first enters Tier 1 for static rule validation. If a high-confidence attack signature is detected, the request is immediately short-circuited and blocked in less than 1ms. Requests passing Tier 1 proceed to Tier 2, where an INT8 ONNX neural classifier calculates a temperature-calibrated risk score. If the neural score exceeds the safety threshold, the payload is blocked. Prompts clearing Tier 2 enter Tier 3 for semantic vector cache screening to verify that the payload does not match historical threat clusters. Prompts clearing all three tiers are verified as safe and dispatched to the primary LLM inference pipeline.

Tier 1: Static Pattern & Structural Heuristic Engine (<1ms)

The initial filter layer consists of deterministic pattern-matching engines and static linear classifiers compiled directly into native binary artifacts³. Implementing the "Mirror" design pattern—where prompt injection data is structured into matched positive and negative cells—enables training sparse character n-gram Support Vector Machines (SVMs)¹². Compiling the learned weight matrices into static Rust binaries allows the system to classify structural payloads in under 0.5 milliseconds without external runtime dependencies¹².

Structural checkers scan for overt jailbreak signatures, such as system prompt exfiltration commands ("ignore previous instructions", "reveal system prompt"), character encoding tricks

(Base64, hex string obfuscation), and homoglyph substitution³. If a Tier 1 rule triggers with a confidence score $S \geq 0.95$, the system immediately halts further evaluation and blocks the request, incurring less than 1ms of total latency overhead³.

Tier 2: Quantized ONNX Neural Inference (5ms – 15ms)

Prompts clearing Tier 1 are passed to an INT8-quantized bidirectional encoder session managed by ONNX Runtime³. Converting PyTorch FP32 models to INT8 ONNX representations reduces weight memory footprints from ~280MB down to ~70MB, allowing full model loading into L1/L2 CPU caches or dedicated GPU memory³. INT8 matrix operations utilize Vector Neural Network Instructions (VNNI) on CPUs or Tensor Cores on GPUs, accelerating matrix multiplications by 3x–4x over FP32 equivalents³.

The ONNX session is initialized once at process launch, establishing persistent memory bindings³. Input text is tokenized via fast C++ SentencePiece tokenizers directly into pre-allocated int64 array buffers (input_ids, attention_mask), eliminating allocation garbage collection pauses³. Single-threaded INT8 ONNX execution of 70M parameter models yields a P50 latency of ~5ms and P95 latency of ~7ms on modern standard server CPUs, sustaining throughputs of ~180 requests/second per core³.

Tier 3: Vector Screening and Semantic Cache Lookup (2ms – 10ms)

To defend against novel variations of known attacks and prevent redundant neural inferences, systems deploy a semantic vector cache alongside the neural classifier¹³. High-dimensional embedding models introduce retrieval latency¹³. By using Matryoshka Representation Learning (MRL) truncated embedding architectures (such as BGE-M3 truncated from 1024 dimensions down to 512 dimensions), vector generation time drops to ~2ms on GPU and ~12ms on CPU while retaining over 95% of native retrieval accuracy¹³.

The prompt embedding vector v_q is queried against an in-memory vector database (e.g., Qdrant, Redis, or FAISS HNSW index) containing pre-indexed malicious attack vectors and safe query hashes¹³. If the maximum cosine similarity $S_{\cos}(v_q, v_{\text{threat}}) \geq \tau_{\text{attack}}$ (typically tuned between 0.88 and 0.92), the prompt is rejected instantly as a known attack variant¹³. If $S_{\cos}(v_q, v_{\text{safe}}) \geq \tau_{\text{safe}}$, the query is flagged as safe, allowing the system to skip downstream checks entirely¹⁴.

Mitigating RAG Threat Vectors and Over-Defense Dynamics

Retrieval-Augmented Generation pipelines introduce a unique threat surface compared to standalone LLM chat applications⁴. Safety guardrails must filter direct user prompts while simultaneously scanning retrieved document chunks before inserting them into the generator's context window⁴.

In a RAG workflow, data filtering operates along two entry points. Direct user prompts pass through the input guardrail cascade (Tiers 1–3). Upon passing, the query triggers vector retrieval against internal knowledge bases. Before retrieved document chunks are concatenated into the main LLM context window, they pass through a secondary context guardrail scan to neutralize embedded exploits. Verified chunks and clean prompts are then safely merged into the generator's prompt context.

Direct vs. Indirect Prompt Injection in RAG Contexts

Direct Prompt Injection (DPI) occurs when a user directly submits adversarial payload instructions designed to subvert system prompts or manipulate tool access rights¹⁸. Conversely, Indirect Prompt Injection (IPI) occurs when adversaries plant hidden instruction strings inside external documents, web pages, vector database entries, or third-party API outputs⁴. When the RAG engine retrieves these poisoned chunks, the main LLM executes the embedded instructions unknowingly, leading to data exfiltration or unauthorized system actions⁴.

Because RAG context windows often aggregate multiple retrieved chunks totaling thousands of tokens, running context checks within a 30ms window requires chunk-level parallelization²⁵. Retrieved text chunks are split into 512-token segments and processed concurrently across parallel ONNX execution threads, preventing total retrieval scanning time from scaling linearly with context length⁶.

The Over-Defense Dilemma: Navigating Trigger Word Biases

A major engineering challenge in automated prompt screening is avoiding over-defense, defined as the false positive classification of benign requests containing risk-correlated terms²⁷. Standard prompt safety models often over-index on specific trigger words (such as ignore, bypass, system, execute, or override)²⁷. When benign enterprise queries legitimately contain these terms, naive classifiers incorrectly flag them as malicious attacks²⁷. Evaluations using specialized over-defense datasets reveal significant performance disparities across safety architectures. The NotInject benchmark provides a diagnostic suite comprising 339 benign prompts enriched with common attack trigger phrases (e.g., "Teach me how to ignore unhelpful code warnings in Python", "Explain SQL injection for exam preparation")²⁷. Uncalibrated baseline prompt guards drop to near-chance accuracy (<60%) on NotInject due to lexical shortcut biases²⁸. Similarly, the XSTest benchmark evaluates exaggerated safety behaviors across safe prompts that touch upon sensitive or risky keywords⁷. Modern guardrail models combat over-defense by implementing data-geometry balancing during training via strategies such as Mitigating Over-Defense for Free (MOF)²⁷. Training pipelines explicitly pair malicious attack inputs with benign samples sharing identical syntax and trigger frequency³. To ensure high system utility in production, operating points must be tuned against strict False Positive Rate (FPR) limits on clean conversation streams³.

Guardrail	Benchmark F1	WildChat /	NotInject	P50 CPU
-----------	--------------	------------	-----------	---------

Model Architecture	Score	LMSYS FPR	Over-Defense Accuracy	Latency
Generative LLM (Llama Guard 3 8B)	0.72 – 0.81 ⁷	3.50% – 5.20%	68.2%	120.0 ms – 350.0 ms ⁸
Decoder CoT (GuardReasoner 8B)	0.831 ⁶	4.10%	71.0%	450.0 ms – 1200.0 ms ⁶
Uncalibrated Encoder (ProtectAI v2)	0.764	4.85% ²⁸	< 60.0% ²⁸	8.0 ms – 12.0 ms
Quantized Encoder (DeBERTa-v3 70M)	0.943 [cite: 3]	1.24% [cite: 3]	88.4% [cite: 3]	5.0 ms [cite: 3]
Lightweight Encoder (LeanGuard 395M)	0.829 [cite: 6, 11]	1.15% [cite: 6, 11]	89.2% [cite: 6, 11]	14.0 ms [cite: 6, 11]

Tuning model operating thresholds to maintain an FPR below **1.25%** prevents high user friction while catching over 95% of active injection attempts³.

High-Performance Engine Optimization Techniques

Executing neural models within microsecond- and single-digit millisecond limits requires explicit low-level runtime optimizations across software frameworks and hardware layers². Python remains the standard language for AI development; however, its Global Interpreter Lock (GIL) and runtime garbage collection introduce unpredictable latency spikes ranging from 10ms to 50ms, violating strict sub-30ms P99 SLOs³. High-performance guardrail services isolate core inference loops within compiled native environments³. Implementing native C++ or Rust service wrappers using frameworks such as Rust axum or C++ gRPC enables direct interface calls to the onnxruntime C API³. Thread pools for inference sessions are pinned directly to physical CPU cores, preventing context-switching overhead and hyper-threading contention². Furthermore, memory for input arrays, token indices, and output probability tensors is pre-allocated during service initialization and reused across requests,

establishing a zero-allocation inference path³.

Virtualized public cloud infrastructure introduces shared hypervisor interrupts and noisy-neighbor network contention, adding up to 30% jitter to tail latency (P99)². Deploying low-latency guardrail nodes on bare-metal compute instances eliminates hypervisor scheduling gaps, stabilizing execution times². Unlike high-throughput batch processing systems, real-time guardrails process incoming requests immediately at a batch size of 1². Requests are evaluated instantly upon ingress rather than waiting in micro-batches, ensuring minimal queue latency². Model weight binaries are loaded into system memory via mmap, permitting shared memory access across process boundaries and enabling sub-millisecond process restarts³.

Comprehensive Comparison of Architectural Strategies

Selecting an optimal guardrail architecture requires balancing latency budgets against memory consumption, threat coverage, and structural implementation complexity³.

Architectural Strategy	P50 Latency	P95 Latency	Memory Footprint	Accuracy (F1)	Threat Vectors Covered	Implementation Complexity
Static Compiled N-Gram (Rust)	0.5 ms [cite: 12]	0.9 ms [cite: 12]	< 15 MB [cite: 12]	0.65 – 0.75	Direct structural injections, known keywords	Low
INT8 ONNX Small Encoder (70M-86M)	5.0 ms [cite: 3]	7.0 ms [cite: 3]	~70 MB [cite: 3]	0.91 – 0.94 [cite: 3]	Direct/Indirect injections, jailbreaks, PII	Medium
Bidirectional Encoder (LeanGuard)	14.0 ms [cite: 6, 11]	22.0 ms [cite: 6, 11]	~400 MB [cite: 6, 11]	0.82 – 0.89 [cite: 6, 11]	Deep semantic harm, multi-policy classification	Medium

395M)					tion	
MRL Semanti c Vector Cache	3.0 ms [cite: 13]	8.0 ms [cite: 14]	~500 MB [cite: 13, 14]	0.85 – 0.90	Rephrase d historical attacks, payload variants	Medium-High
Small Decoder LLM (Llama Guard 1B)	40.0 ms ²²	120.0 ms ³¹	~2.0 GB ³¹	0.75 – 0.82	Broad safety policy complian ce	High
CoT Reasonin g Guard (GuardR easoner 8B)	450.0 ms ⁶	1200.0 ms ⁶	~16.0 GB ⁶	0.83 – 0.85 ⁶	Complex multi-tur n logical violations	High

Strategic Implementation Roadmap

Implementing a sub-30ms safety guardrail within an enterprise RAG pipeline requires a structured deployment sequence spanning optimization, calibration, and pipeline integration phases³.

The deployment process begins with Model Quantization and Runtime Setup. System engineers select a dedicated bidirectional encoder discriminator (such as DeBERTa-v3-xsmall or Prompt-Guard 2 86M)³. PyTorch weights are exported to INT8-quantized ONNX representations using dynamic or static range quantization, compressing model footprints to ~70MB³. The quantized ONNX engine is then embedded inside a compiled C++ or Rust service wrapper to eliminate Python interpreter overhead and lock contention³.

Next, the pipeline undergoes Temperature Calibration and Thresholding. A balanced validation corpus containing both clean traffic and trigger-enriched benign prompts (e.g., NotInject) is

routed through the native service layer³. The scalar temperature T is optimized via Negative Log-Likelihood minimization to ensure calibrated probability outputs³. Classification thresholds

($\tau_{\text{attack}} \geq 0.50$, $\tau_{\text{high_risk}} \geq 0.85$) are set to maintain a False Positive Rate below 1.25%, preventing false alarms on benign enterprise queries³.

Finally, the architecture enters Cascaded Pipeline Integration. The three-tier screening chain is wired directly inline with the API gateway³. Ingress prompts trigger Tier 1 static Rust rules for sub-millisecond pattern filtering³. Prompts passing Tier 1 undergo Tier 2 single-pass ONNX neural evaluation (5ms–7ms) alongside Tier 3 MRL vector cache lookups (3ms–8ms)³. Concurrently, retrieved RAG context chunks are chunked into 512-token blocks and processed in parallel across Tier 2 threads before entering the generator's context window⁴. Adopting this multi-tiered, discriminative architectural blueprint enables enterprise RAG applications to enforce strict safety policies, filter prompt injections, and block compromised retrieval content inline, consistently operating well within the hard 30ms latency budget³.

Works cited

1. Low Latency AI Guardrails for Safer GenAI Apps - Alice.io, <https://alice.io/blog/low-latency-ai-guardrails>
2. Lowest Latency AI Inference Provider for Open-Source LLMs (2026 Engineering Guide), <https://www.gmicloud.ai/en/blog/lowest-latency-ai-inference-provider-for-open-source-llms-2026-engineering-guide>
3. Bastion Prompt Protection — Developer Documentation, <https://docs.bastionsoft.com/>
4. Walled Protect - Prompt Injection Defense | WalledAI, <https://walled.ai/platform/walled-protect>
5. guardion/ModernGuard-1 - Hugging Face, <https://huggingface.co/guardion/ModernGuard-1>
6. [2606.26686] Do Safety Guardrails Need to Reason? LeanGuard: A Fast and Light Approach for Robust Moderation - arXiv, <https://arxiv.org/abs/2606.26686>
7. Benchmarking guardrail models for safety, refusal, and latency - Artificial Analysis, <https://artificialanalysis.ai/articles/guardrail-safety-benchmark>
8. Guarded Query Routing for Large Language Models - arXiv, <https://arxiv.org/html/2505.14524v1>
9. Performance - Guardrails AI, <https://guardrailsai.com/guardrails/docs/concepts/performance>
10. LLM Latency Benchmark by Use Cases in 2026 - AIMultiple, <https://aimultiple.com/llm-latency-benchmark>
11. Do Safety Guardrails Need to Reason? LeanGuard: A Fast and Light Approach for Robust Moderation - arXiv, <https://arxiv.org/html/2606.26686v1>
12. Daily Papers - Hugging Face, <https://huggingface.co/papers?q=multi-round%20event%20injection>
13. Semantic Caching for LLM Inference: GPTCache, Redis Vector Cache, and Prompt Cache Setup (2026) | Spheron Blog, <https://www.spheron.network/blog/semantic-cache-llm-inference-gpu-cloud/>
14. What Is a Semantic Cache for LLMs? FutureAGI Guide (2026), <https://futureagi.com/glossary/semantic-cache/>
15. (PDF) LanG -- A Governance-Aware Agentic AI Platform for Unified Security

Operations,

https://www.researchgate.net/publication/403605625_LanG_--_A_Governance-Aware_Agentic_AI_Platform_for_Unified_Security_Operations

16. bastion-prompt-protection Changelog - Safety,
<https://data.safetycli.com/packages/pypi/bastion-prompt-protection/changelog?page=2&>
17. Changelog · Directories by Enterprise DNA,
<https://enterprisedna.co/directories/changelog/>
18. nealtnaidoo/prompt-guard: Adversarial-grade prompt injection detection and content sanitisation middleware for LLM applications - GitHub,
<https://github.com/nealtnaidoo/prompt-guard>
19. google/gemma-3-270m Free Chat Online - Skywork,
<https://skywork.ai/blog/models/google-gemma-3-270m-free-chat-online/>
20. What Is Semantic Caching for LLMs? A 2026 Guide - Future AGI,
<https://futureagi.com/blog/what-is-semantic-caching-llms-2026/>
21. Semantic Caching vs Prompt Caching: You Probably Need Both | Katlextech Blog,
<https://katlex.com/blog/semantic-caching-vs-prompt-caching>
22. Best Open-Source AI Guardrail Models for Enterprise in 2026 | Seven Labs,
<https://www.sevenlabs.site/blogs/best-open-source-ai-guardrail-models-enterprise-2026>
23. Indirect Prompt Injection in RAG Systems and AI Agents: The Attack Nobody Sees Coming, <https://aquilax.ai/blog/indirect-prompt-injection-rag-agents>
24. AI Security Glossary: 35 Key Terms Every Security Team Needs to Know - Repello AI, <https://repello.ai/blog/ai-security-glossary>
25. llm_firewall_1.md · John6666/forum3 at main - Hugging Face,
https://huggingface.co/datasets/John6666/forum3/blob/main/llm_firewall_1.md
26. Adversarial Prompt Evaluation: Systematic Benchmarking of Guardrails Against Prompt Input Attacks on LLMs - arXiv, <https://arxiv.org/html/2502.15427v1>
27. PIGuard: Prompt Injection Guardrail via Mitigating Overdefense for Free - ResearchGate,
https://www.researchgate.net/publication/394270263_PIGuard_Prompt_Injection_Guardrail_via_Mitigating_Overdefense_for_Free
28. InjecGuard: Benchmarking and Mitigating Over-defense in Prompt Injection Guardrail Models - arXiv, <https://arxiv.org/html/2410.22770v1>
29. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models | Request PDF - ResearchGate,
https://www.researchgate.net/publication/382627021_XSTest_A_Test_Suite_for_Identifying_Exaggerated_Safety_Behaviours_in_Large_Language_Models
30. Onnx runtime 1000x slower in c++ than python : r/cpp_questions - Reddit,
https://www.reddit.com/r/cpp_questions/comments/yk3300/onnx_runtime_1000x_slower_in_c_than_python/
31. A Review on LLMs for IoT Ecosystem: State-of-the-art, Lightweight Models, Use Cases, Key Challenges, Future Directions - TechRxiv,
<https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.174844330.01320055>